Models and Tools for Cyber-Physical Systems

Chapter 7: Timed Model

Instructors: Suman Sourav, Jonathan Julian Huerta y Munive
{sumansourav, jjhymuni}@cs.aau.dk

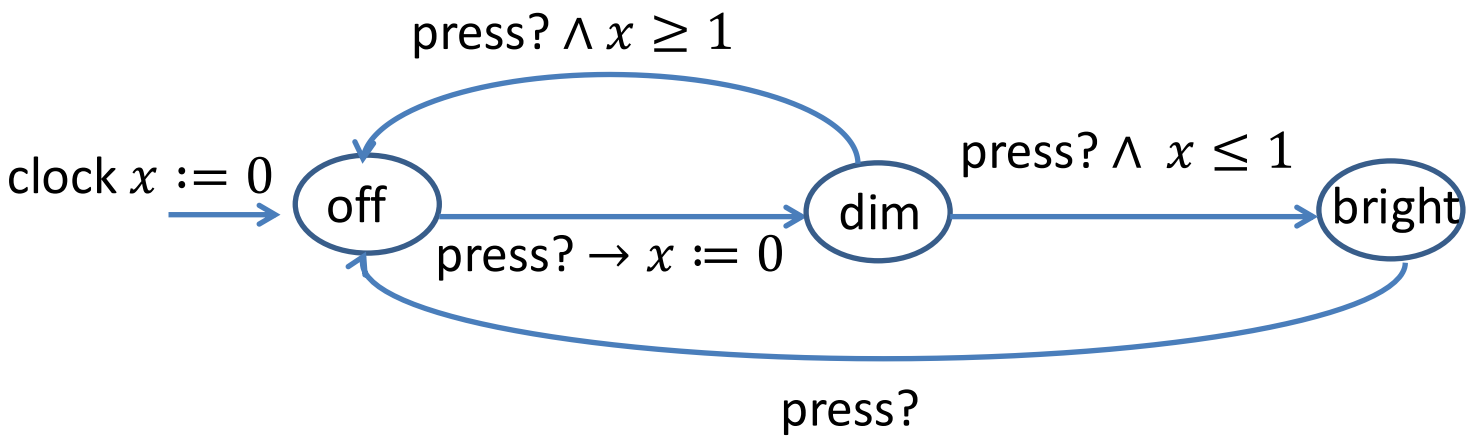
Slides courtesy of Rajeev Alur
alur@cis.upenn.edu

Models of Reactive Computation

- Synchronous model
  - Components execute in a sequence of discrete rounds in lock-step
  - Computation within a round: Execute all tasks in an order consistent with precedence constraints
- Asynchronous model
  - Speeds at which different components execute are independent
  - Computation within a step: Execute a single task that is enabled
- Timed model
  - Like asynchronous for communication of information
  - Can rely on global time for coordination

Quantitative aspects
(timing, energy, bandwidth, memory, …)
crucial for several embedded systems

Example Timed Model



Initial state: $(mode = \text{off}, x = 0)$; or short: $(\text{off}, 0)$

Timed transition: $(\text{off}, 0) \xrightarrow{0.5} (\text{off}, 0.5)$

Input transition: $(\text{off}, 0.5) \xrightarrow{\text{press?}} (\text{dim}, 0)$

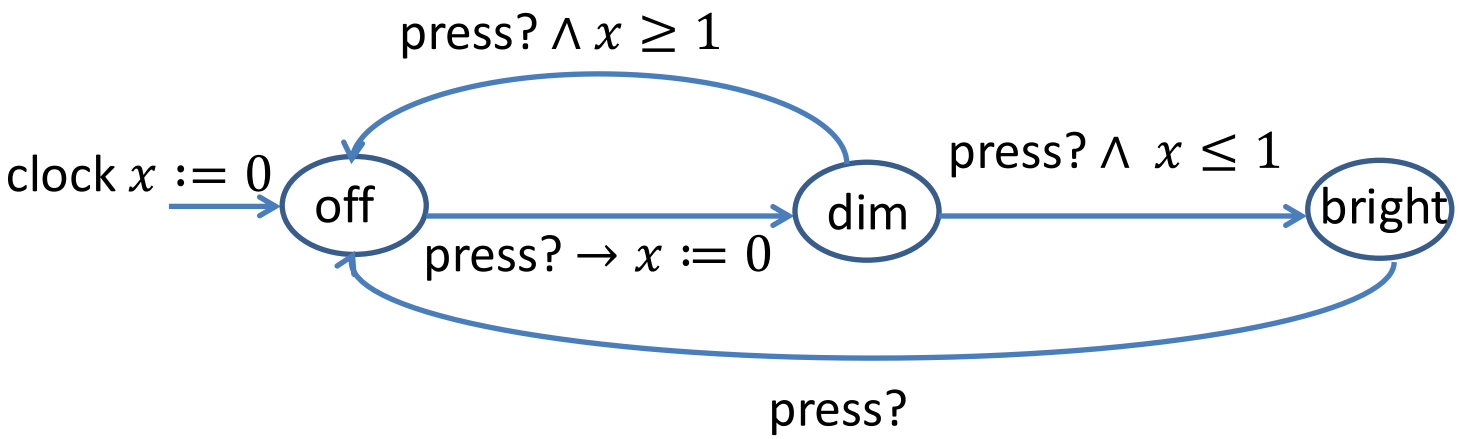
Timed transition: $(\text{dim}, 0) \xrightarrow{0.8} (\text{dim}, 0.8)$

Input transition: $(\text{dim}, 0.8) \xrightarrow{\text{press?}} (\text{bright}, 0.8)$; $alternative$:

Timed transition: $(\text{dim}, 0.8) \xrightarrow{1} (\text{dim}, 1.8)$

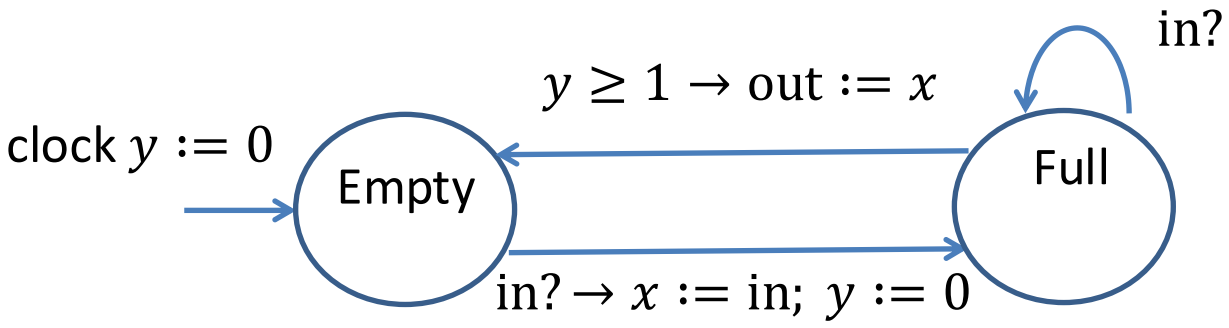
Input transition: $(\text{dim}, 1.8) \xrightarrow{\text{press?}} (\text{off}, 1.8)$

Example Timed Model



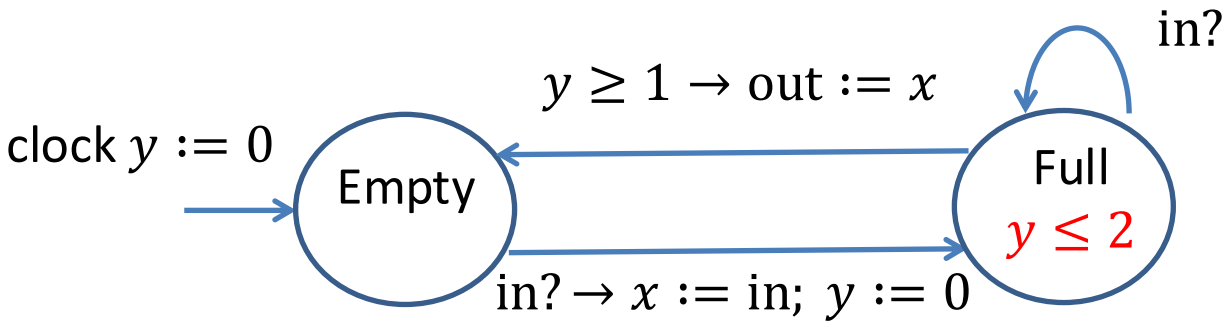
- Clock variables
  - Tests and updates in mode switches like other variables
  - New feature: During a timed transition of duration δ, the value of a clock variable increases by an amount equal to δ
- Timing constraint: Setting x to 0 for "off → dim" and guard $x \leq 1$ for "dim → bright" specifies that timing of these two transitions is at most 1 time unit apart

Modeling Timed Buffer



- Mode says whether the buffer is full (Full) or not (Empty)
- State variable x remembers the last input value when buffer is full
- Clock variable y tracks the time elapsed since buffer got full
- When full, input events are ignored
- Guard $y \geq 1$ ensures that at least 1 time unit elapses in mode Full
- How to ensure that the mode switch from Full to Empty is executed before the clock y exceeds the upper bound 2?

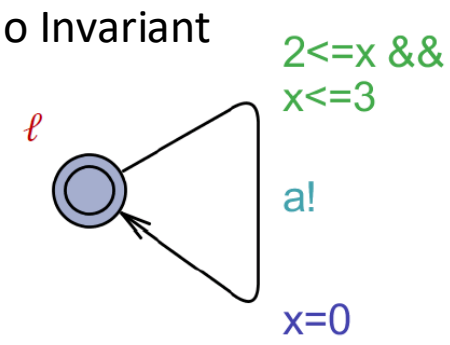
Clock Invariants



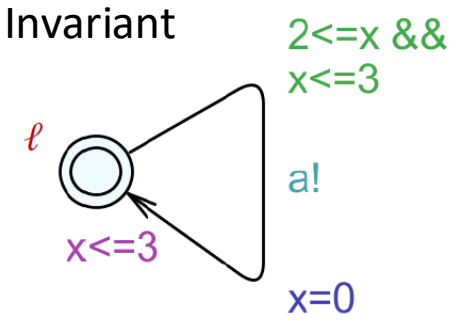
- The constraint "$y \leq 2$" associated with the mode Full is called "clock invariant"
- A timed transition of duration δ is allowed only when the clock invariant remains true during the entire duration
  - Timed transition $(\text{Full}, x = 1, y = 0.8) \xrightarrow{0.7} (\text{Full}, x = 1, y = 1.5)$ allowed
  - Timed transition $(\text{Full}, x = 1, y = 0.8) \xrightarrow{1.4} (\text{Full}, x = 1, y = 2.2)$ disallowed
- Clock invariants are useful to limit how long a process stays in a mode

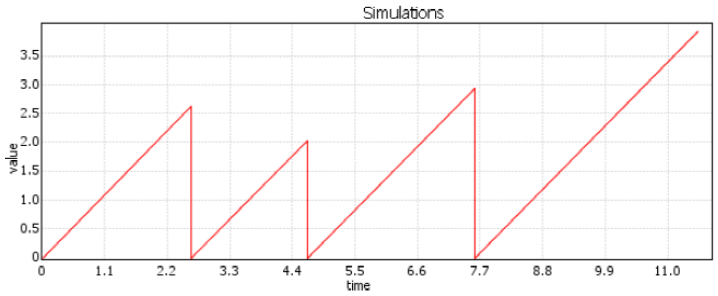
Invariants or Not?

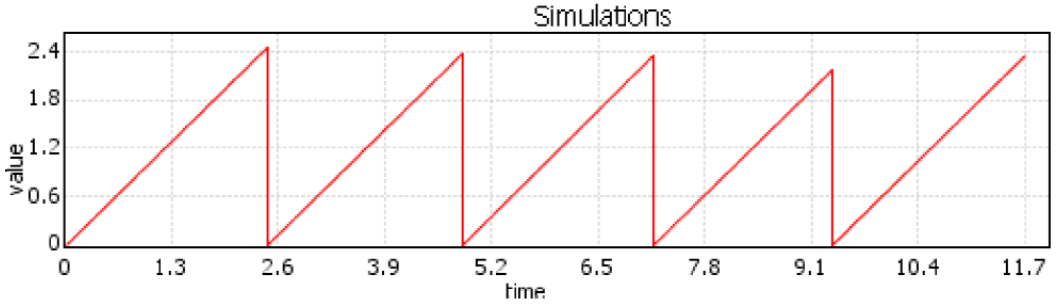
No Invariant



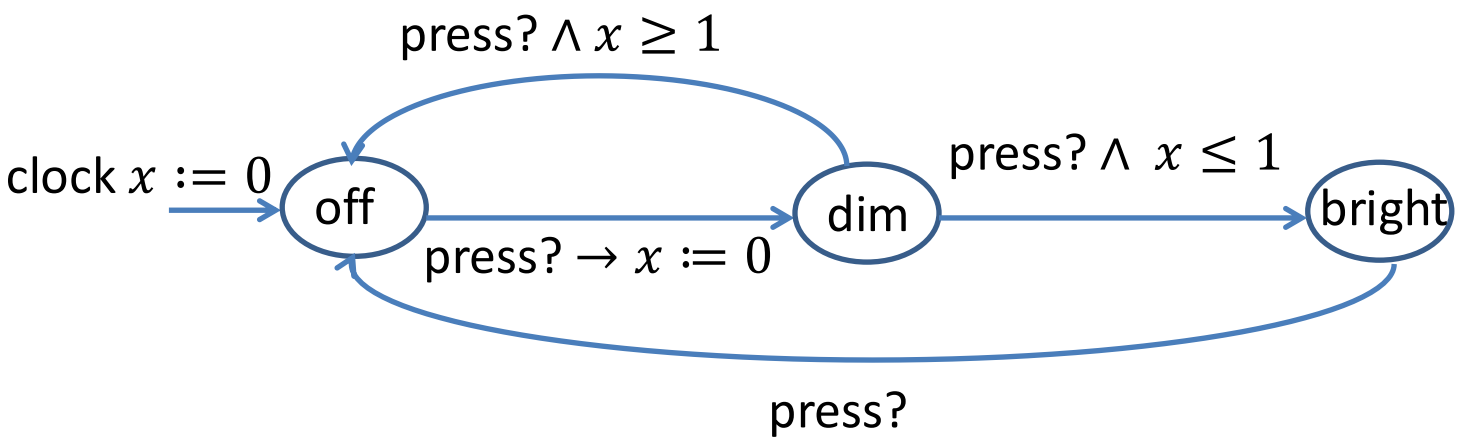
Invariant







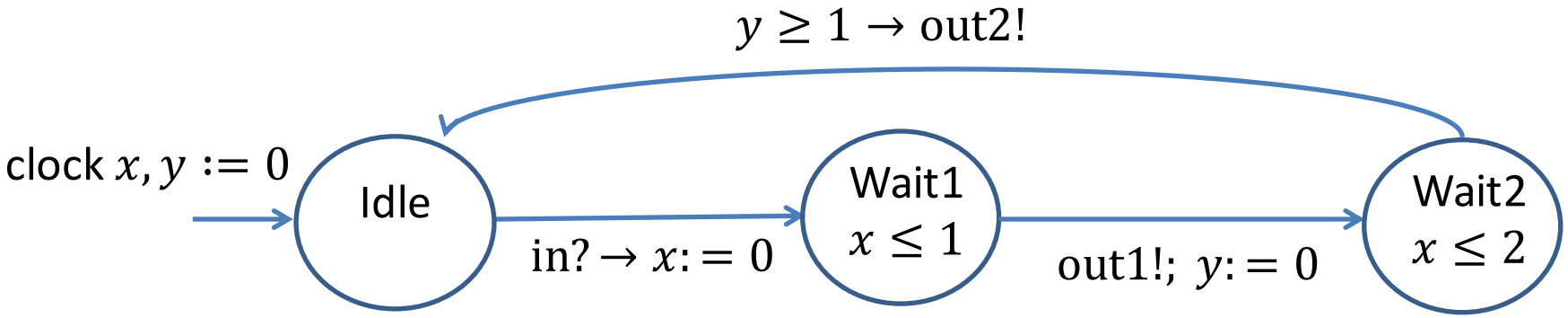
Exercise: Smart Light Switch



- Current model relies upon the user to switch the light off.
- Exercise:

  Adjust the model such that the light switches off from bright mode automatically after 120 time units of no press from the user.

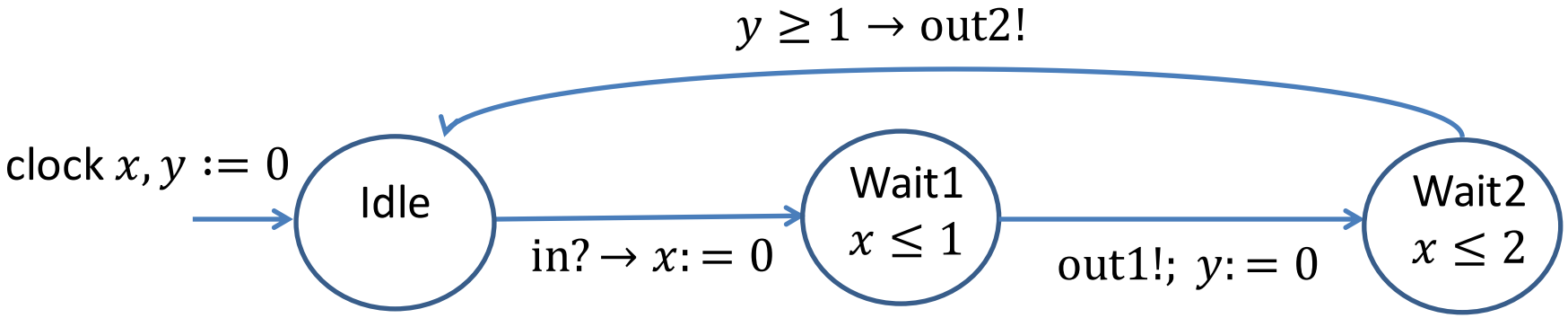
Example with Two Clocks (1)



- Input event in; Output events out1, out2
- Two clock variables x, y
- As time elapses, both clocks increase at the same rate
- Sample timed transitions:

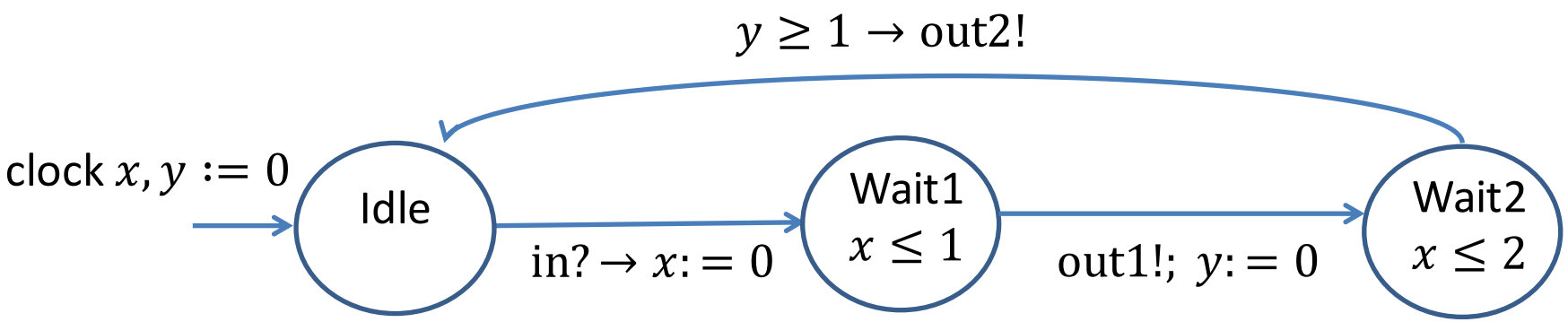
$$(\text{Wait2}, x = 0.8, y = 0) \xrightarrow{0.3} (\text{Wait2}, 1.1, 0.3) \xrightarrow{0.72} (\text{Wait2}, 1.82, 1.02)$$

Example with Two Clocks (2)



- Clock x tracks time elapsed since the occurrence of input event
- Clock y tracks time elapsed since the occurrence of output event out1
- What is the behavior of this model?
- If input event occurs at time t, the process issues an output on channel out1 at time t' within the interval $[t, t+1]$, and then issues another output on channel out2 at time t'' within the interval $[t' + 1, t + 2]$

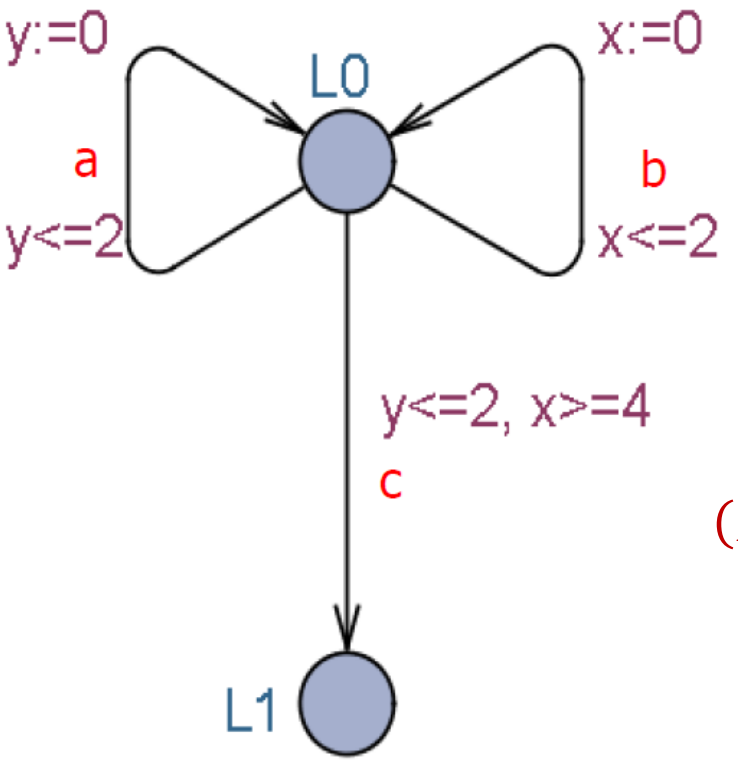
Execution



- Initial state: all clocks have value 0
- Two types of transitions
  - Discrete transition: change mode, no time passes
  - Timed transition: time passes uniformly, stay in mode

$$(Idle, 0, 0) \xrightarrow{0.3} (Idle, 0.3, 0.3) \xrightarrow{in?} (\text{Wait1}, 0, 0.3) \xrightarrow{0.6} (\text{Wait1}, 0.6, 0.9)$$
$$\xrightarrow{out1!} (\text{Wait2}, 0.6, 0) \xrightarrow{1.3} (\text{Wait2}, 1.9, 1.3) \xrightarrow{out2!} (Idle, 1.9, 1.3)$$

Another Example with Two Clocks



$$(L_0, x = 0, y = 0) \xrightarrow{2} (L_0, x = 2, y = 2) \xrightarrow{a} (L_0, x = 2, y = 0) \xrightarrow{2} (L_0, x = 4\ y = 2) \xrightarrow{c}$$

Is L1 reachable ?

Example Specification (1)

Consider a timed process with

Input event i, Output events u and v

Desired behavior:

Whenever it receives input, it produces both its output events

Time delay between $i?$ and $u!$ is in the interval $[2, 4]$

Time delay between $i?$ and $v!$ is in the interval $[3, 5]$

Input events received during these times are ignored

Exercise: design the timed process satisfying this specification

Example Specification (2)

Consider a timed process with

Input event i, Output events u and v

Desired behavior:

Whenever it receives input, it produces both its output events

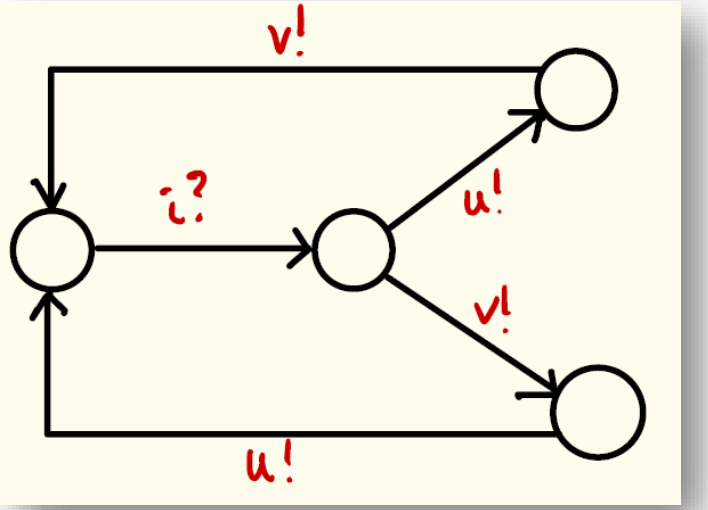
Time delay between $i?$ and $u!$ is in the interval $[2, 4]$

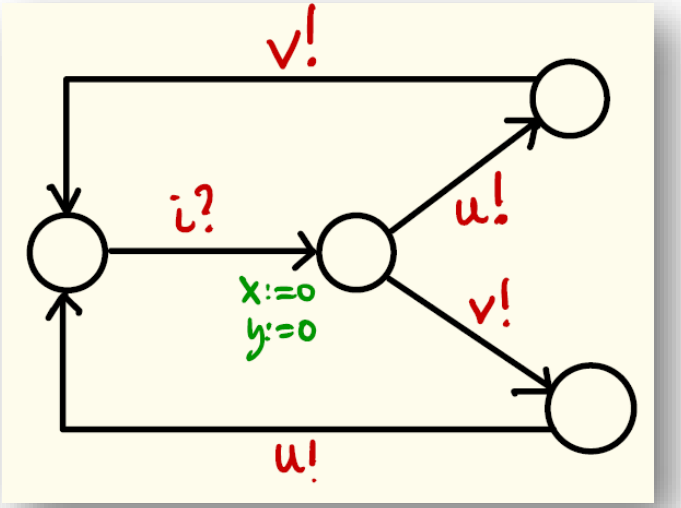
Time delay between $i?$ and $v!$ is in the interval $[3, 5]$

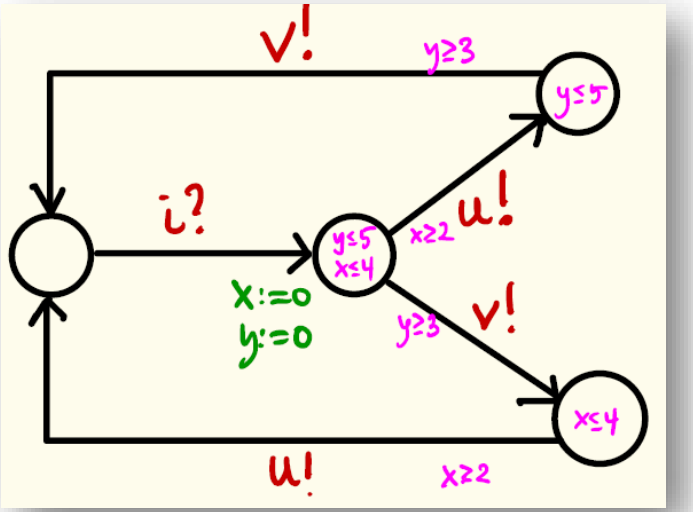
Input events received during these times are ignored

Clock x measures time delay between $i?$ and $u!$

Clock y measures time delay between $i?$ and $v!$







Example Specification (3)

Consider a timed process with

Input event i, Output events u and v

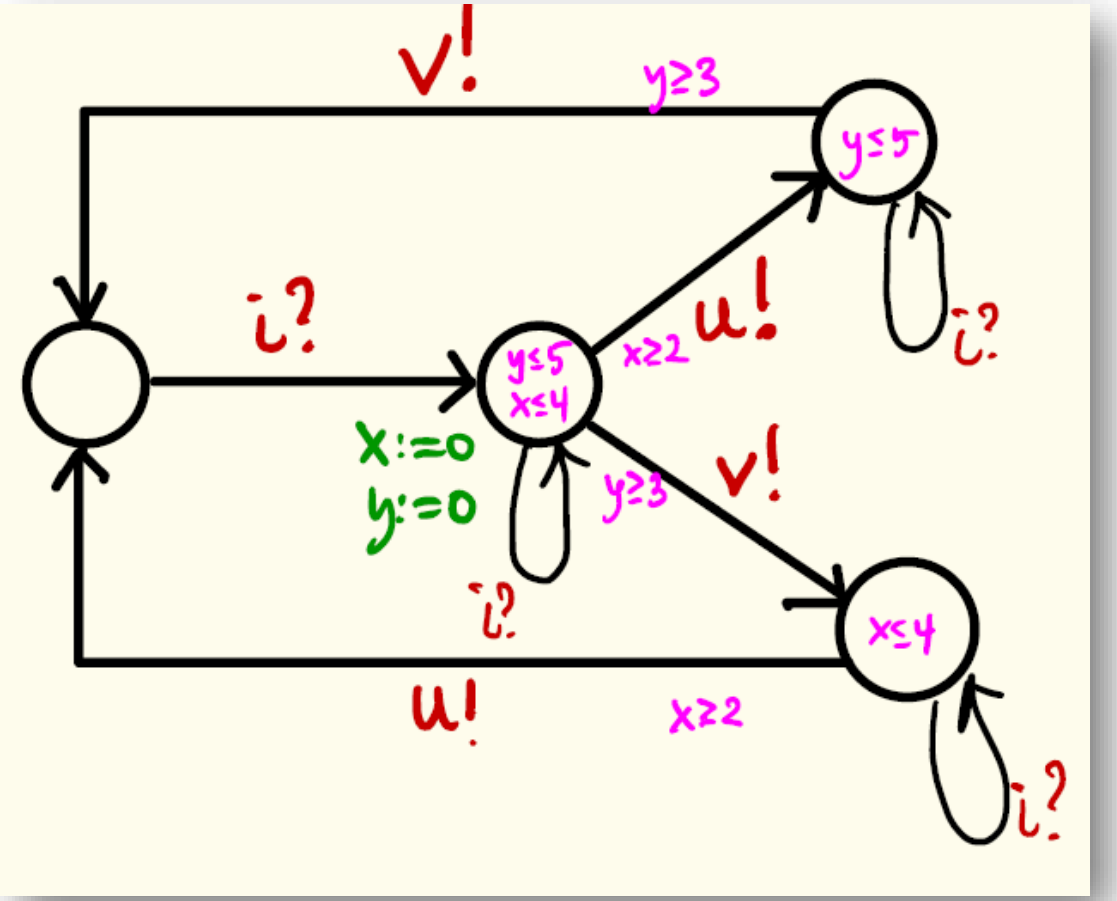
Desired behavior:

Whenever it receives input, it produces both its output events

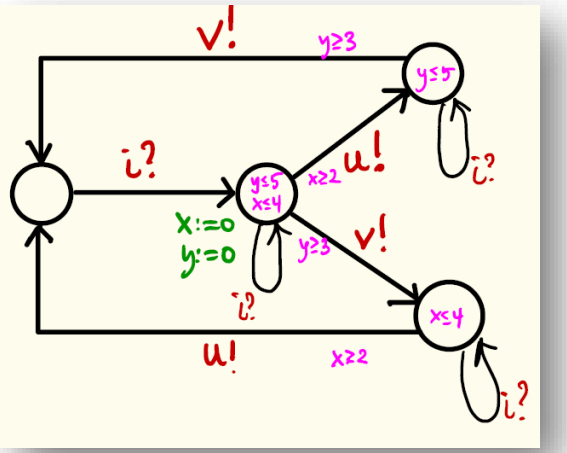
Time delay between $i?$ and $u!$ is in the interval $[2, 4]$

Time delay between $i?$ and $v!$ is in the interval $[3, 5]$

Input events received during these times are ignored



Definition of Timed Process



- A timed process TP consists of
  1. An asynchronous process P, where some of the state variables can be of type clock (ranging over non-negative real numbers)
  2. A clock invariant $\boldsymbol{CI}$, which is a Boolean expression over the clock variables of P
- Inputs, outputs, states, initial states, internal actions, input actions, and output actions are defined exactly as in the asynchronous model
- Timed actions: Given a state s and real-valued time $\delta > 0$, $\boldsymbol{s \xrightarrow{\delta} s + \delta}$ is a timed transition of duration δ if the state $s + t$ satisfies the expression CI for all values of t in the interval $[0, \delta]$
  - For a state s and time $\boldsymbol{\delta}$, $\boldsymbol{s + \delta}$ is another state which assigns the value $s(x) + \delta$ to every clock variable x, and $s(y)$ to every variable y of a different type (i.e., not clock)
  - If clock invariant is a convex constraint, then it is sufficient to check that the end-states s and $s + \delta$ satisfy CI

Composition of Processes

Example: Timed Buffer

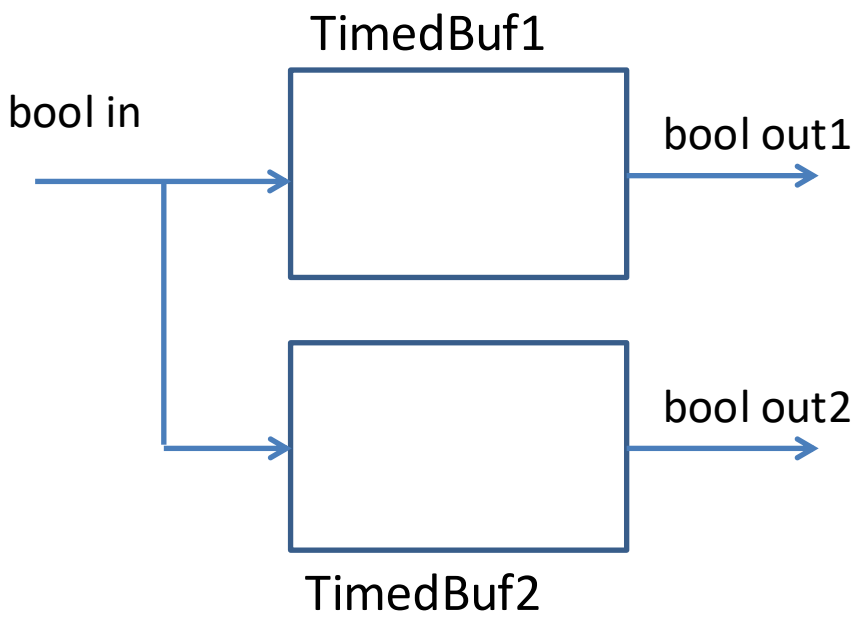
- Buffer with a bounded delay

- Behavior to be modeled: Input received on the channel in is transmitted on the output channel out after a delay of δ such that

$$LB \leq \delta \leq UB$$

(i.e., we know lower/upper bounds on this delay)

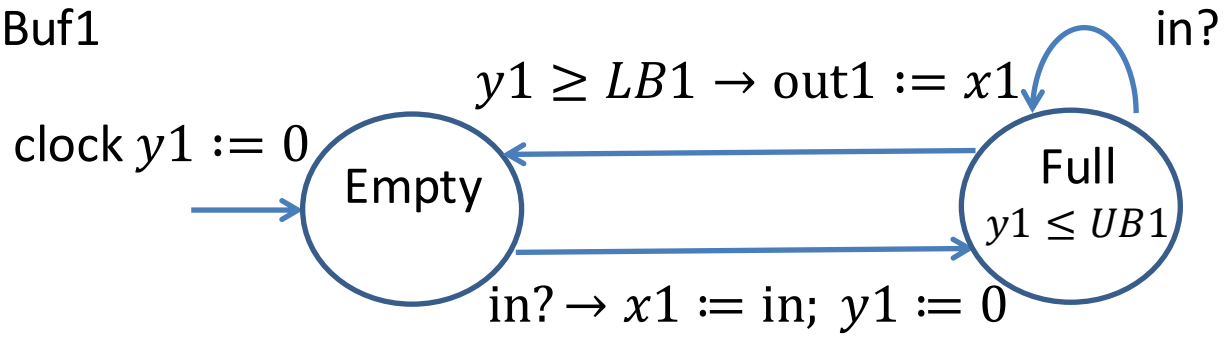
Composition of Processes



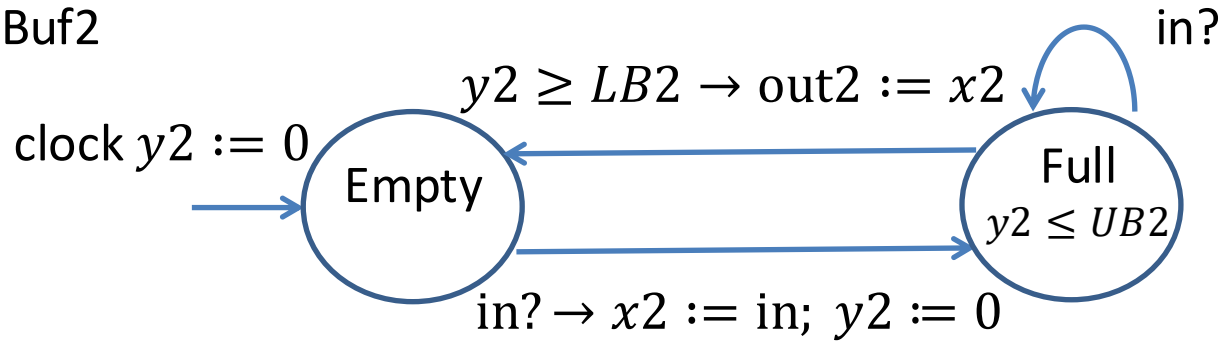
- How to construct timed process corresponding to the composition of the two processes?
- What are the possible behaviors of the composite process?

Composition of Timed Processes

TimedBuf1

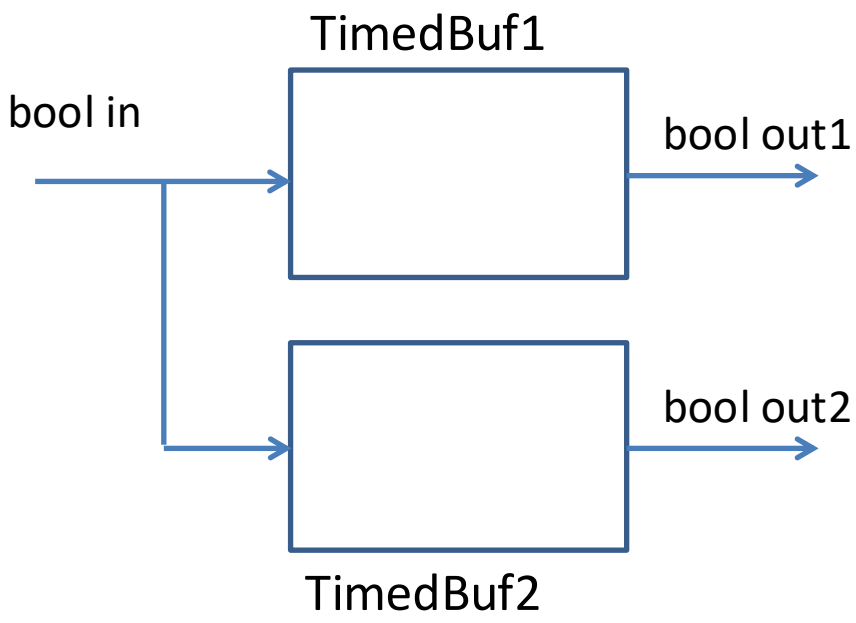


TimedBuf2



Construct the composite process with four modes

Composition of Processes



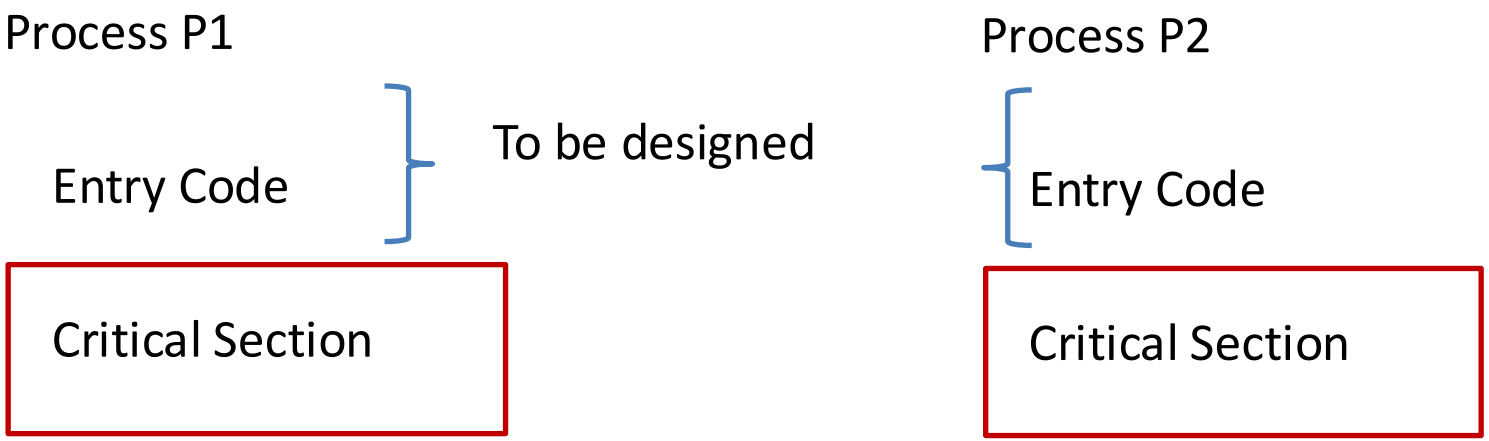
- If $UB1 < LB2$, then out1 guaranteed to occur before out2
  - Implicit coordination based on bounds on delays
- Is it possible to observe 2 out1 events without intervening out2 event?
  - Depends on relative magnitudes of bounds (need timing analysis!)

Definition of Parallel Composition

- Consider timed processes $TP1 = (P1, CI1)$ and $TP2 = (P2, CI2)$
- When is the parallel composition $TP1 \mid TP2$ defined?
  - Exactly when the asynchronous parallel composition $P1 \mid P2$ is defined (that is, when the outputs of the two are disjoint)
- $TP1 \mid TP2 = (P1 \mid P2, CI1 \wedge CI2)$
  - Asynchronous composition of $P1$ and $P2$ defines the internal, input and output actions of the composition
  - Conjunction of the clock invariants defines the clock invariant of the composition
- Consequence: The composite process can allow a timed action of duration δ exactly when both $TP1$ and $TP2$ are willing to wait for time δ

Case Study: Mutual Exclusion

Mutual Exclusion Problem



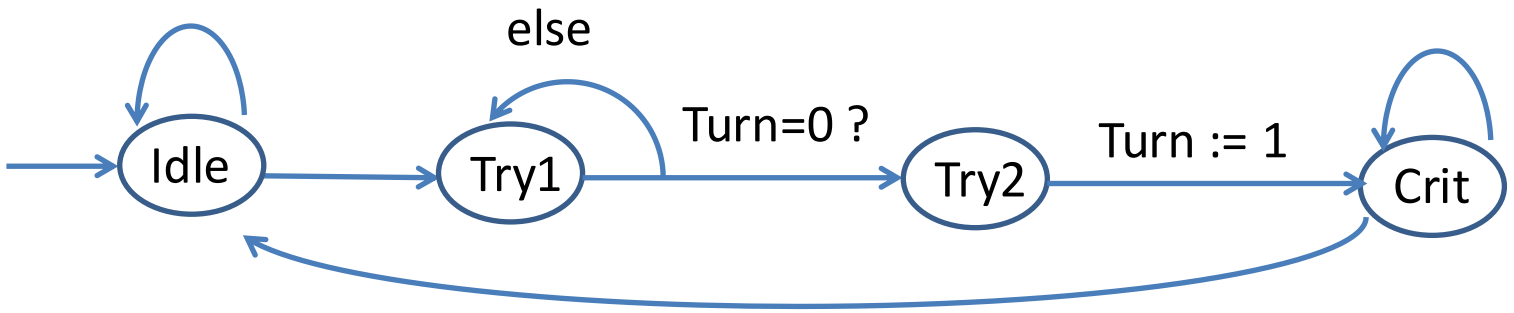
- Safety requirement: Both processes should not be in critical section simultaneously (can be formalized as an invariant)

- Absence of deadlocks: If a process is trying to enter, then some process should be able to enter

Mutual Exclusion: Attempted Solution

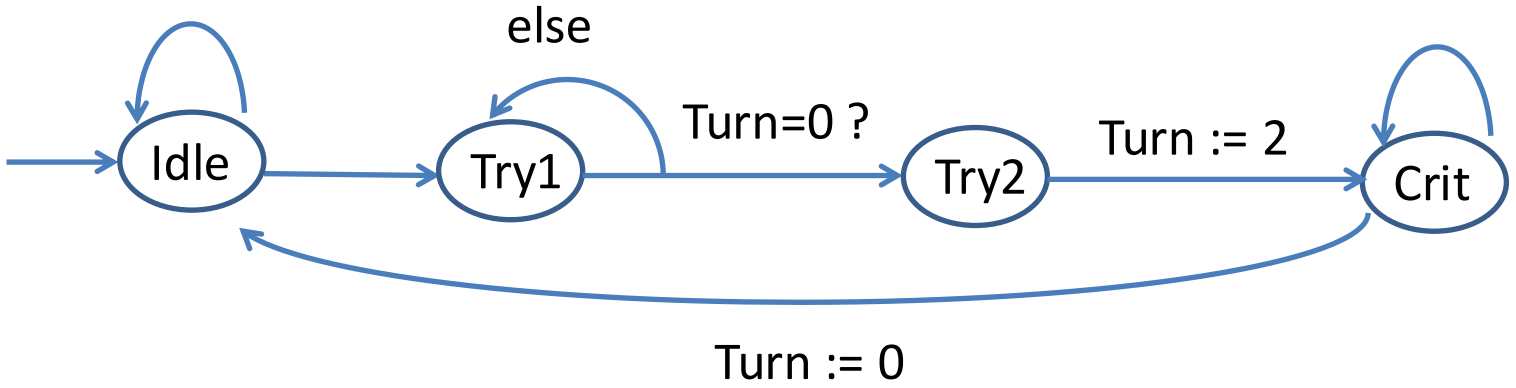
AtomicReg {0, 1, 2} Turn := 0

Process P1



What is the problem?

Process P2

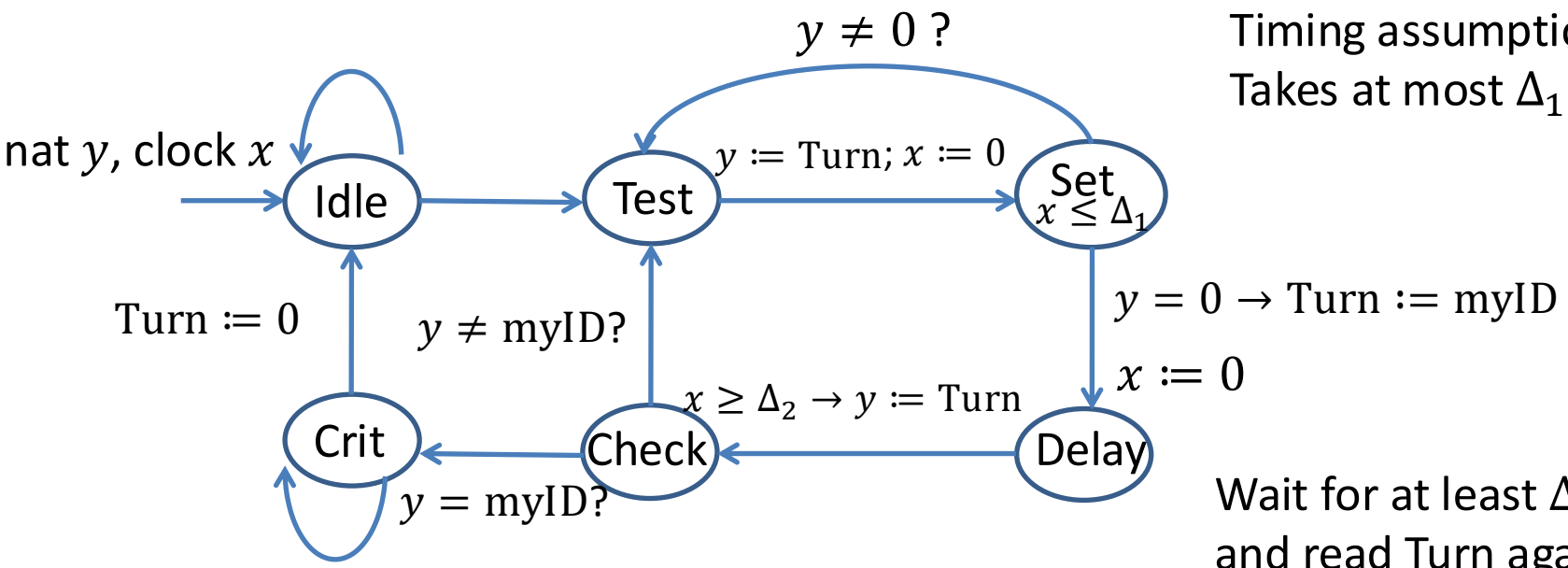


Timing-Based Mutual Exclusion

1. When a process wants to enter the critical section, it reads the shared variable Turn
2. If Turn $\neq 0$ then try again (go to step 1)
3. If Turn $= 0$ then set Turn to your ID
4. Proceeding directly to critical section is a problem (since the other process may also have concurrently read Turn to be 0, and updating Turn to its own ID)
5. Solution: Delay and wait until you are sure that concurrent writes are finished
6. Read Turn again: if Turn equals your own ID then proceed to critical section, if not, go to step 1 and try again
7. When done with critical section, set Turn back to 0

Fischer's Protocol for Mutual Exclusion

AtomicReg Turn := 0



Timing assumption:
Takes at most Δ_1 to write

Wait for at least Δ_2 time,
and read Turn again

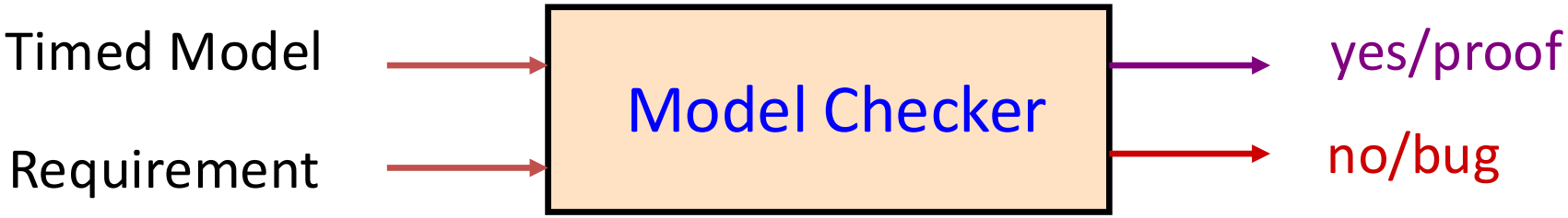
Why does it work ?

Properties of Fischer's Protocol

- Assuming $\Delta_2 > \Delta_1$, the algorithm satisfies:
  - Mutual exclusion: Two processes cannot be in critical section simultaneously
  - Deadlock freedom: If a process wants to enter critical section, then some process will enter critical section
- Protocol works for arbitrarily many processes (not just 2)
  - Note: In the asynchronous model, mutual exclusion protocol for N processes is a lot more complex than Peterson's algorithm
- Does the protocol satisfy the stronger property of starvation freedom: If a process P_i wants to enter the critical section, then it eventually will?
- If $\Delta_2 \leq \Delta_1$, then does mutual exclusion hold? Does deadlock freedom hold?

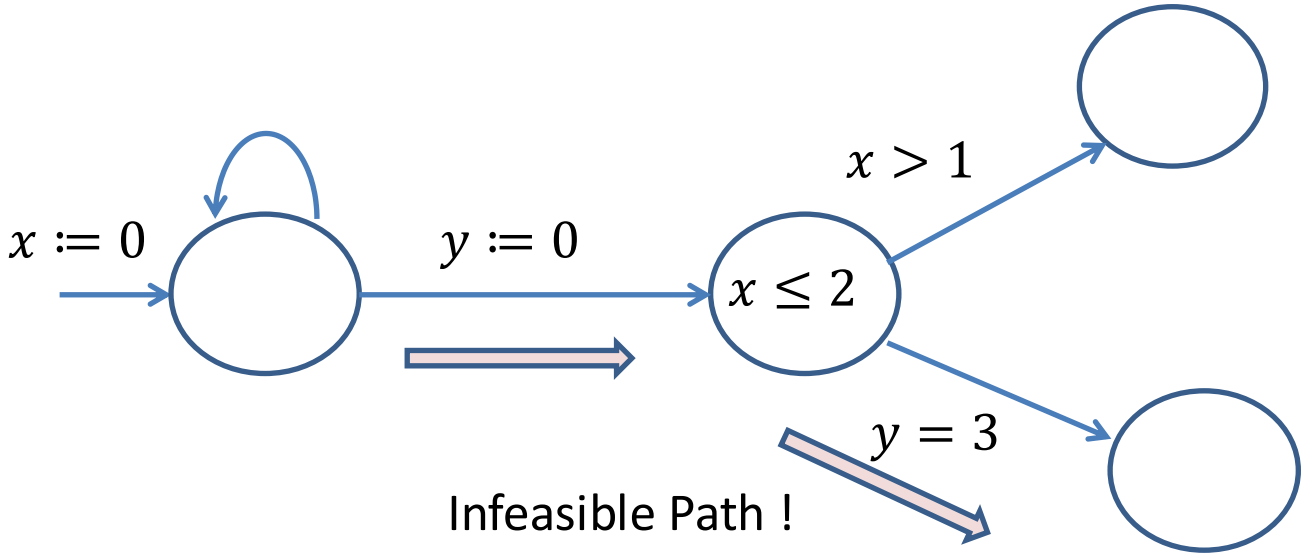
Timing Analysis

Timing Analysis



- How to adapt algorithms for searching through the state-space of a model in presence of clock variables and timing constraints?
- Application: Formal analysis of timing-based coordination and communication protocols, and many other systems …
- Must handle the space of clock valuations symbolically!
- Popular model checker: Uppaal (according to Rajeev Alur)

Timing Analysis Example



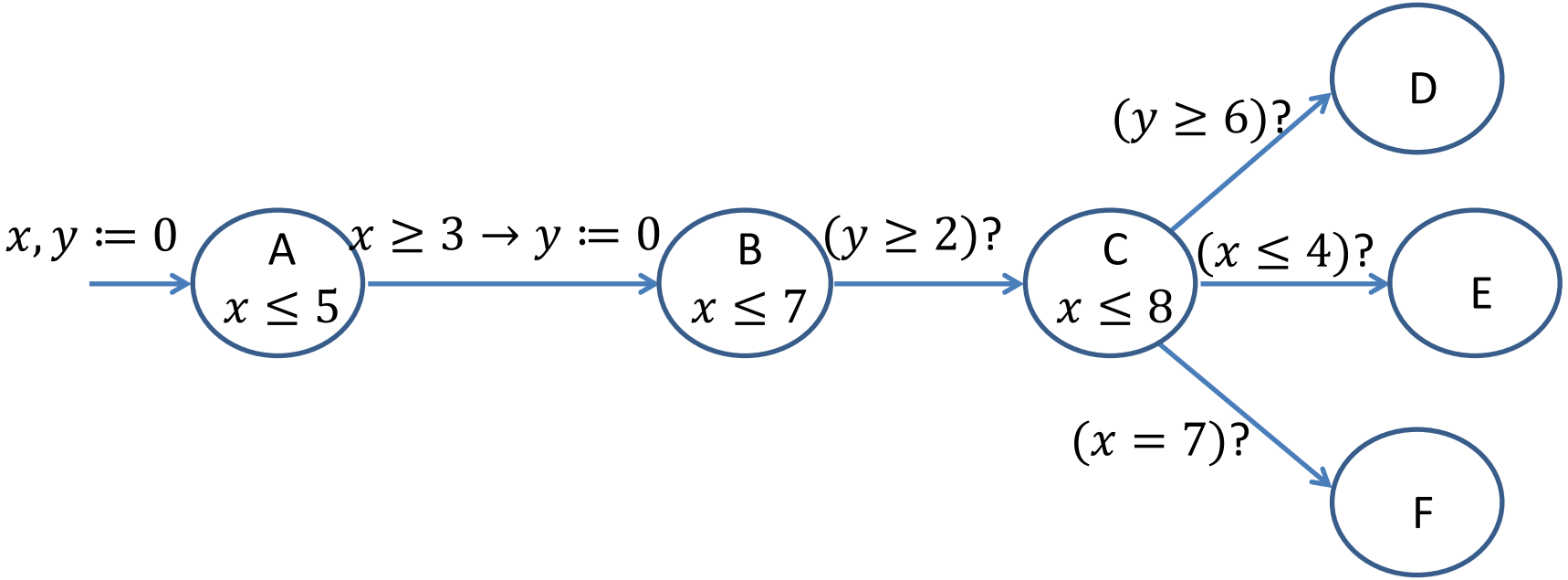
Timed Automata

- Motivation: When is exact analysis of timing constraints possible?
- A timed process TP is a timed automaton if for every clock variable x
  - Assignments to x in the description of TP are of the form $x := 0$ (called reset)
  - An atomic expression involving x in the description of TP (in clock invariants or in guards) must of the form "$x \approx k$", where k is a constant and $\approx$ is a comparison operation ($=, \leq, <, >, \geq$)
- Closed under parallel composition: If TP_1 and TP_2 are timed automata, then $TP_1 \mid TP_2$ is also a timed automaton
- In such a model, one can express constant lower and upper bounds on timing delays
- Finite-state timed automaton: A timed automaton where all variables other than clock variables have finite types (e.g. Boolean, enumerated)
  - State space is still infinite due to clock variables, but verification is possible

Note on Terminology

- The terminology used in the field of timed automata is slightly different from the one used in the book (on which these slides are based)
- The circles in drawings are called modes in the book, while they are called locations in timed automata
- The state of an automaton is (l, v), with l a location and v a clock valuation (an assignment of values to the clock variables)

Timing Analysis Example

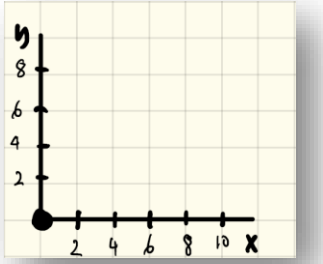


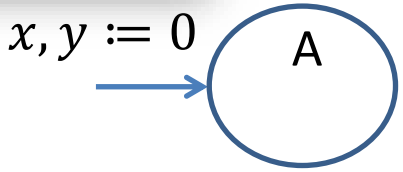
Which of the locations/modes D, E, F are reachable?

Requires propagation of the reachable combinations of x and y symbolically

Timing Analysis Example

R0



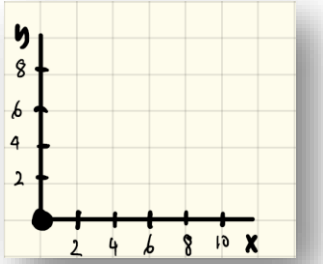


Initial set of clock-valuations: $x = 0 \wedge y = 0$

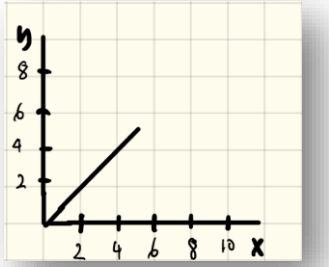
Clock zone: Uniform representation of constraints that arise during analysis

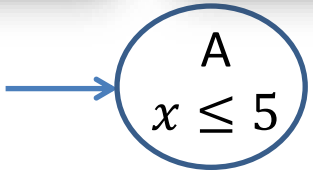
Timing Analysis Example

R0



R1





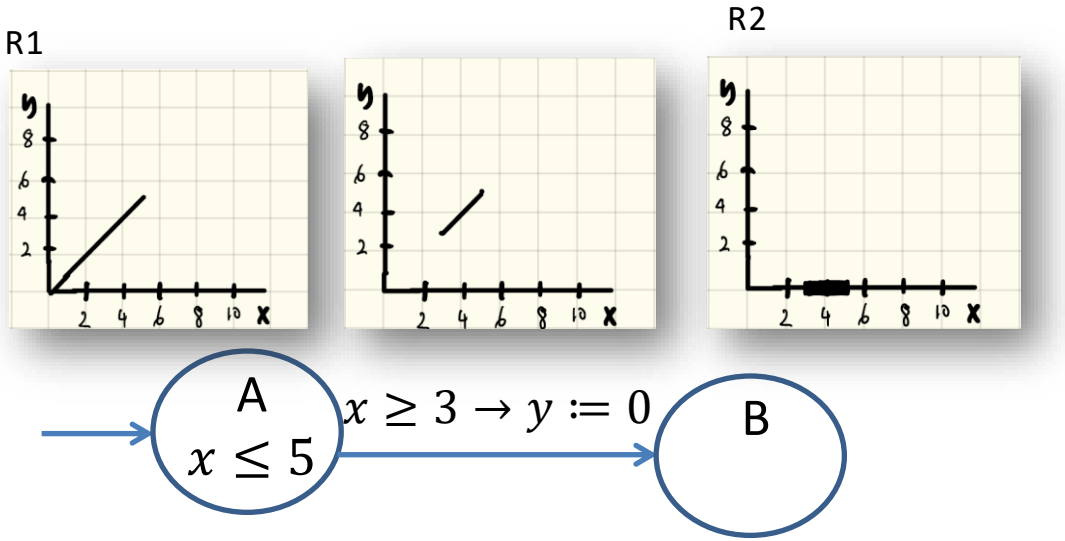
Starting from a state in R_0, as time elapses, which clock valuations are reachable?

During a timed transition, values of all clocks increase

Compute effect of timed transitions observing the clock invariant

note that different clocks increase with the same speed (diagonally)

Timing Analysis Example



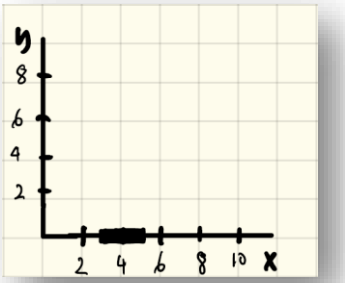
Desired clock zone R_2: What are the clock valuations upon entry in B?

Intersect guard $x \geq 3$ with the clock zone R_1

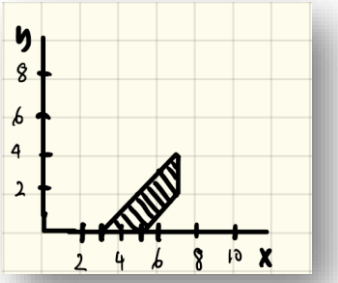
Capture the effect of assignment $y := 0$ (project on x-axis)

Timing Analysis Example

R2



R3

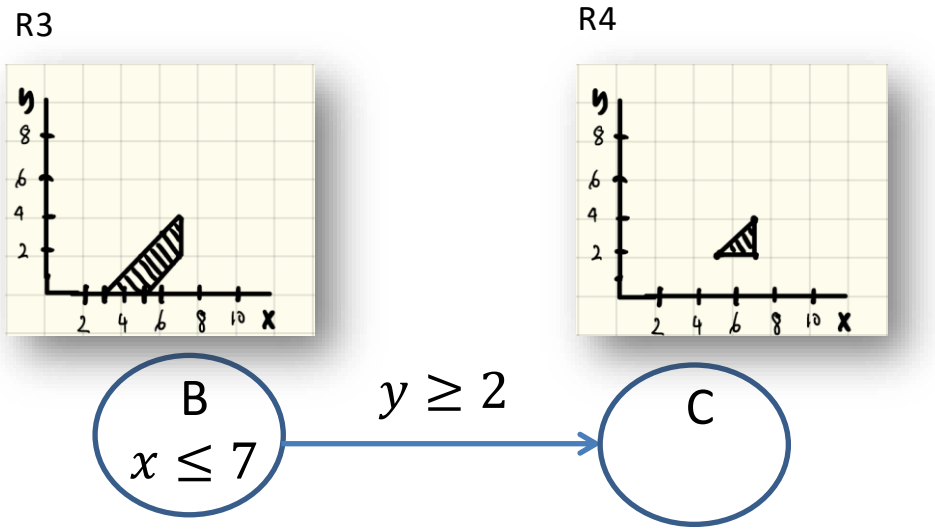


B

$x \leq 7$

Starting from a state in R_2, as time elapses, which clock valuations are reachable?

Timing Analysis Example

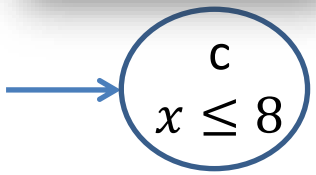


Compute set of clock valuations upon entry in C
i.e., intersect guard $y \geq 2$ with clock zone R_3

Timing Analysis Example

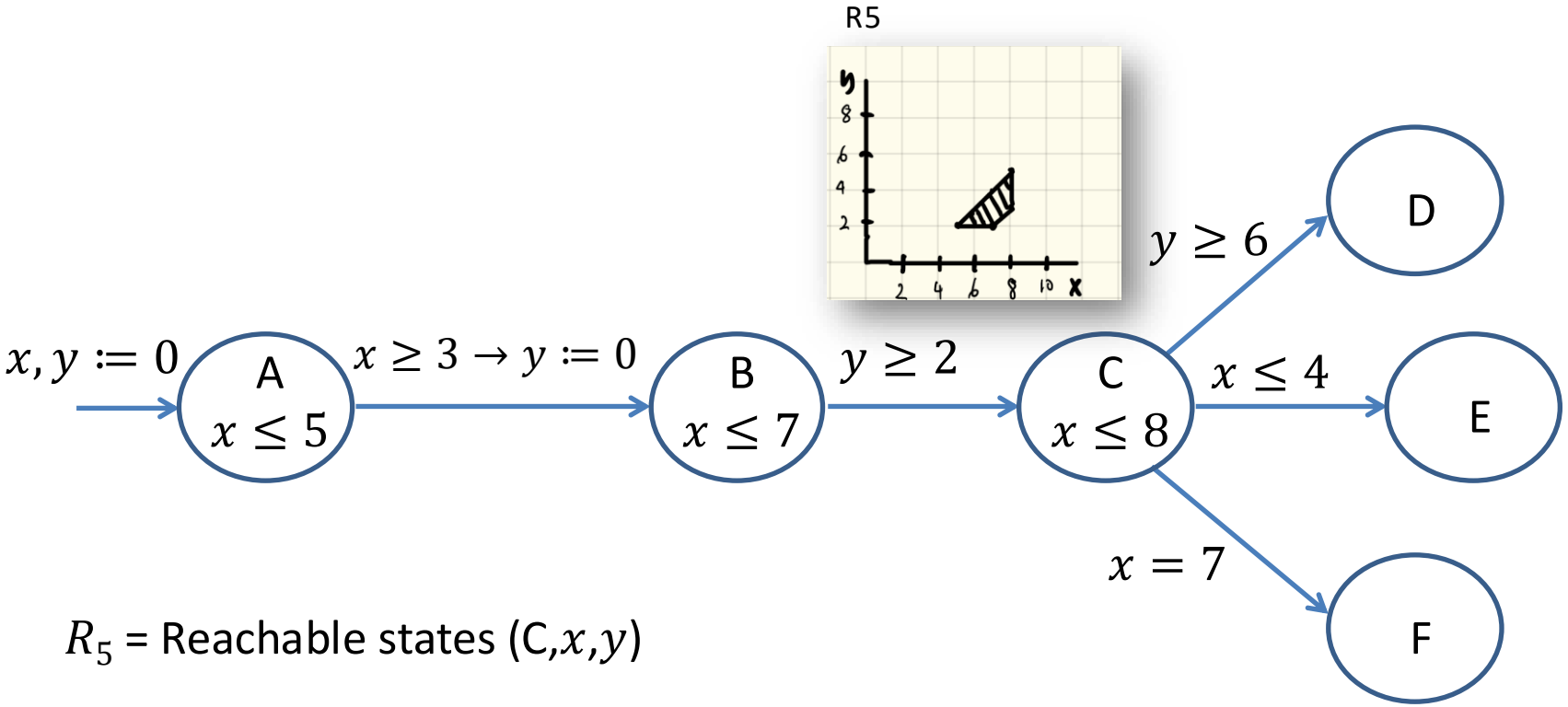
R4

R5



Compute set of clock valuations reachable due to timed transitions in mode C

Timing Analysis Example



R_5 = Reachable states (C,x,y)

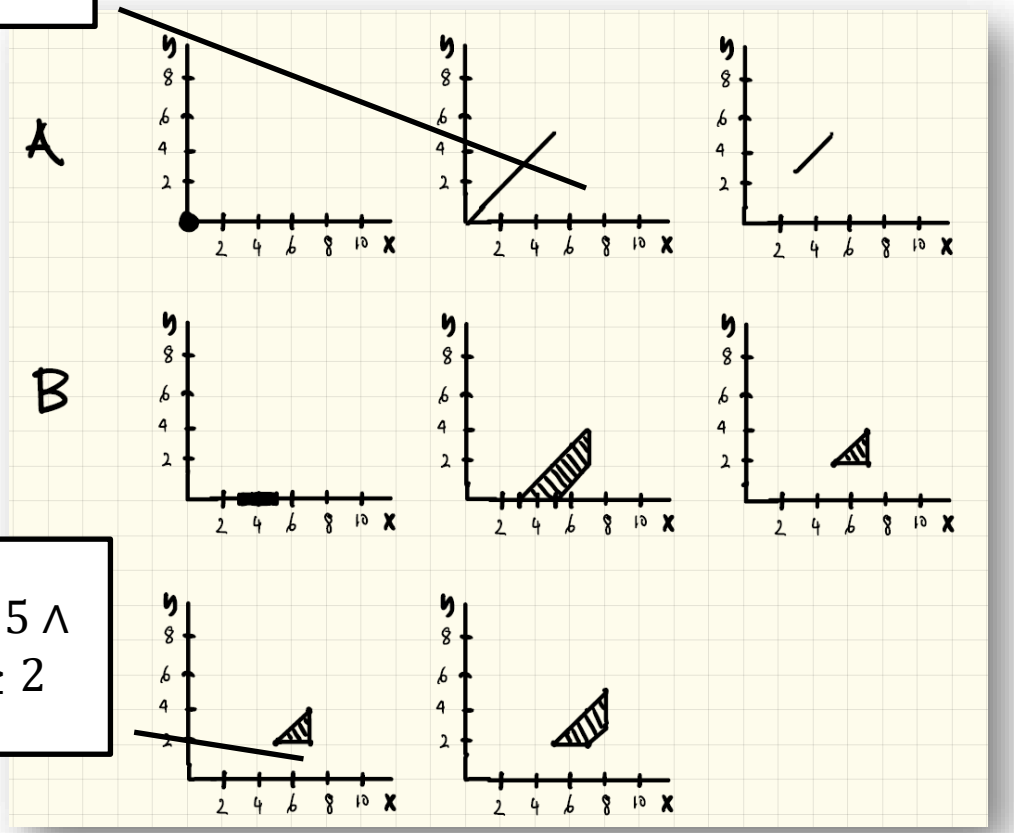
Intersection of R_5 and $y \geq 6$ is unsatisfiable, i.e., mode D not reachable

Intersection of R_5 and $x \leq 4$ is unsatisfiable, i.e., mode E not reachable

Intersection of R_5 and $x = 7$ is satisfiable, i.e., mode F is reachable

Symbolic Data Structures for Zones

$$0 \le x - y \le 0 \ \wedge \ x \le 5$$



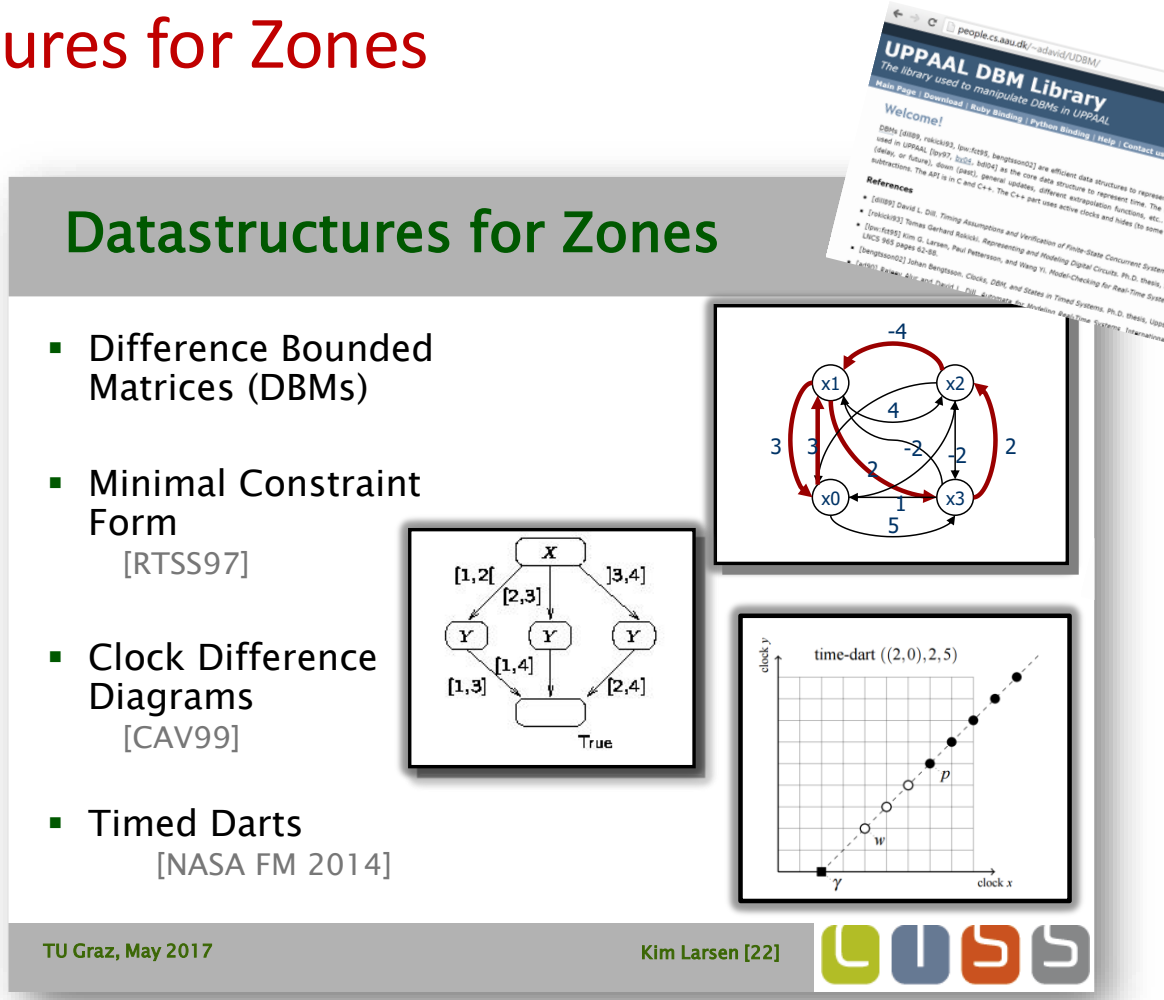
$$3 \le x - y \le 5 \wedge x \le 7 \wedge y \ge 2$$

Zones

= conjunction of constraints

$x \sim n$ and $x - y \sim n$

= difference constraints



Efficient Representations and Operations